

Assignment 1 Report

Liam Watson

April 29, 2026

1 Introduction

This report details answers to key questions posed throughout the Digital Watch Part 1 Assignment and analyses all waveforms given by each module using a Testbench. We aim to highlight key findings and confirm all output behaves as expected after testing.

Repository `fpga-digital-watch` forked and `seven_segment` module completed locally. Changes committed and pushed as backup to GitHub.

2 Report 1

Report 1

Explain why Verilator accepts `if (blank)` but not `if (ACTIVE_LOW)`.

Figure 1: Screenshot taken from `Digital_Watch_Part_1.pdf`

Verilator only accepts `blank` otherwise the signal `blank` is not used in the instance `seven_segment`.

3 Report 2

Report 2

For every module, include a screenshot of the waveforms and briefly explain why they are correct, pointing to specific features in the traces.

Figure 2: Screenshot taken from `Digital_Watch_Part_1.pdf`

3.1 Seven Segment Display

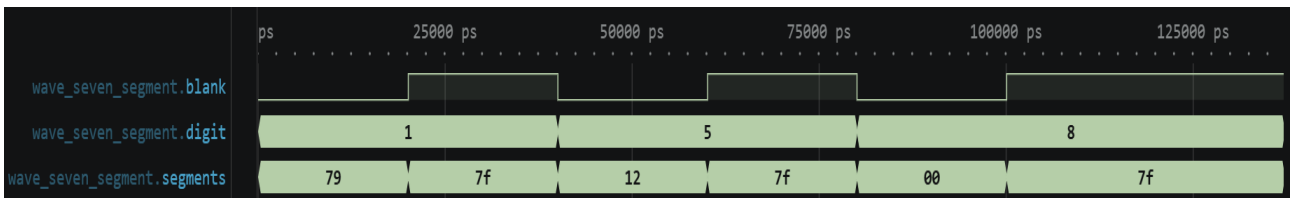


Figure 3: Measured waveform for `seven_segment.sv`

The waveform for `seven_segment` displays inputs `blank`, `digit`, and output `segments`. We observe that:

- When `blank` is low `segments` corresponds to the appropriate display output (e.g. when `digit` = 1, `segments` = 79 or 7'b1111001).
- When `blank` is high `segments` is set to 7F or 7'b1111111 which turns off all segments.

3.2 Parameterised Up-Down Counter with Enable

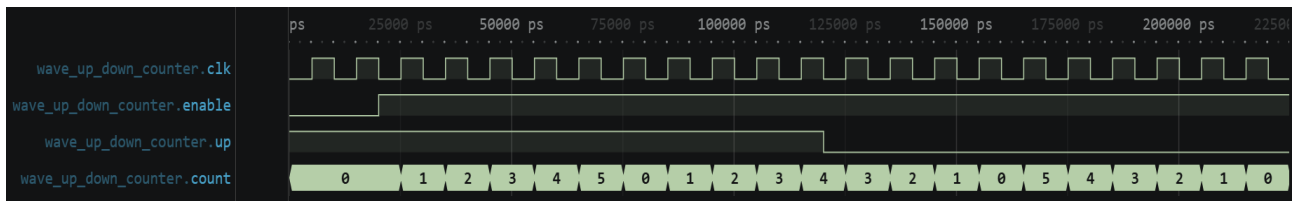


Figure 4: Measured waveform for up_down_counter.sv

The waveform displays inputs **clk**, **enable**, **up** and output **count**. **MAX** is set to 5 and **WIDTH** set to 3 in our Testbench. We observe that:

- When **enable** is low **count** holds its current value at 0.
- When **enable** and **up** are high **count** increments and wraps after **count** = 5.
- When **enable** is high and **up** is low **count** decrements and wraps after **count** = 0.

3.3 Binary to Binary-Coded Decimal

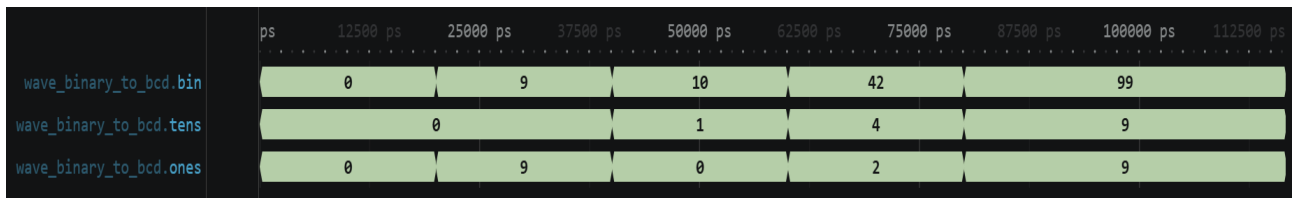


Figure 5: Measured waveform for binary_to_bcd.sv

The waveform displays inputs **bin**, and outputs **tens**, and **ones**. We observe that:

- **bin** displays input digit.
- **tens** corresponds with 10s column of **bin**.
- **ones** corresponds with 1s column of **bin**.

3.4 Cascaded Hour-Minute-Second Counter with Enable

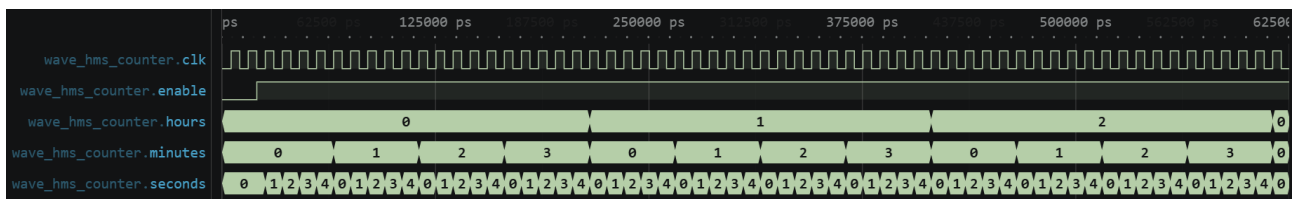


Figure 6: Measured waveform for hms_counter.sv

The waveform displays inputs **clock**, **enable** and outputs **hours**, **minutes** and **seconds**. We observe that:

- When **enable** is low all outputs hold current value.
 - When **enable** is high **seconds** increments every rising clock edge.
 - **minutes** increments after **seconds** reaches its max value.
 - **hours** increments after **minutes** reaches its max value.
 - each counter wraps after its max value is reached.
-
- **bin** displays input digit.
 - **tens** corresponds with 10s column of **bin**.
 - **ones** corresponds with 1s column of **bin**.

3.5 Modulo N Counter

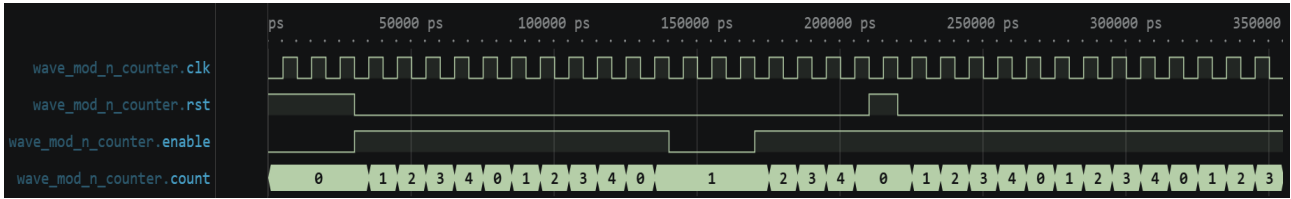


Figure 7: Measured waveform for mod_n_counter.sv

The waveform displays inputs **clk**, **rst**, and **enable** and output **count**. We observe that:

- When **enable** is low all outputs hold current value.
- When **rst** is high **count** is reset to 0 and has priority over **enable**.
- When **enable** is high and **rst** is low **count** increments every rising clock edge.
- **count** wraps after **N - 1** is reached.

3.6 Restartable Rate Generator

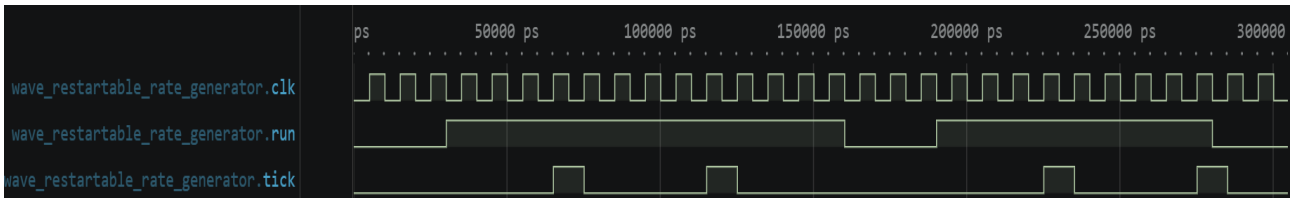


Figure 8: Measured waveform for restartable_rate_generator.sv

The waveform displays inputs **clk**, and **run**, and output **tick**. We observe that:

- When **run** is low **tick** is low.
- When **run** is high for **CYCLE_COUNT - 1** rising edges **tick** outputs a one cycle pulse.

3.7 Top Time Display V1

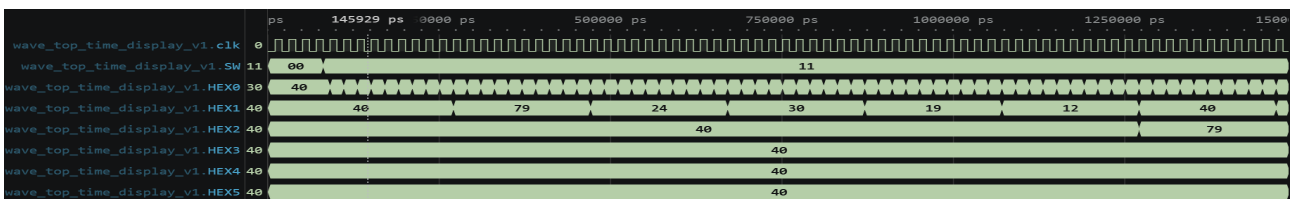


Figure 9: Measured waveform for top_time_display_v1.sv

The waveform displays inputs **clk**, and **SW**, and outputs **HEX0**, **HEX1**, **HEX2**, **HEX3**, **HEX4**, and **HEX5**. We observe that:

- When **SW** is 00 counter holds 00:00:00.
- When **SW** is high second counter increments every cycle and minutes increments to 1 when seconds rolls over from 59 to 0.

3.8 Pulse Width Generator

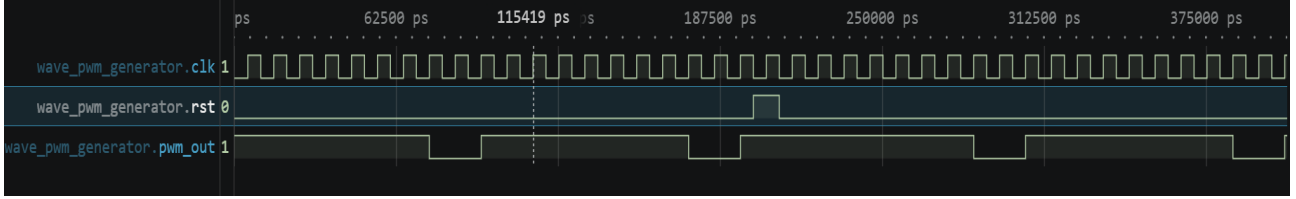


Figure 10: Measured waveform for `pwm_generator.sv`

The waveform displays inputs `clk`, and `rst`, and output `pwm_out`. We observe that:

- When `pwm_out` has period of **PERIOD_CYCLES** and is high for first [**DUTY_CYCLES**].
- When `rst` is high `pwm_out` stays high next clock edge.

3.9 Rising Edge Detector

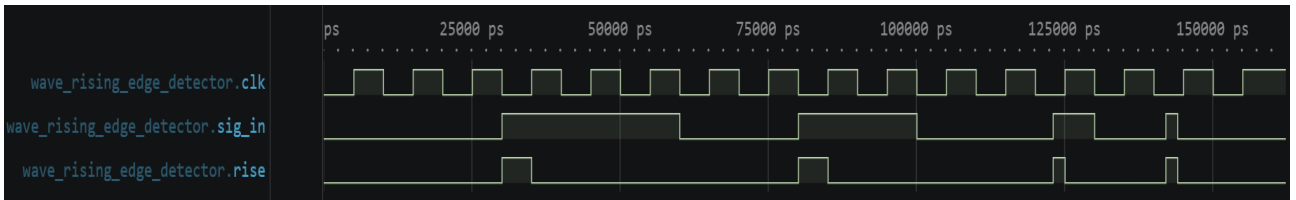


Figure 11: Measured waveform for `rising_edge_detector.sv`

The waveform displays inputs `clk`, and outputs `sig_in` and `rise`. We observe that:

- When `sig_in` is high `rise` is high for remainder of clock cycle.

3.10 Button Hold Detect

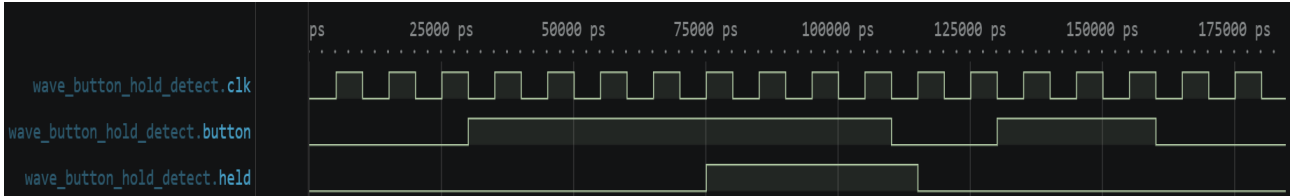


Figure 12: Measured waveform for `button_hold_detect.sv`

The waveform displays inputs `clk` and `button` and output `held`. We observe that:

- `held` goes high after `button` is pressed for **HOLD_CYCLES** only.
- `held` goes low after button has been released.

3.11 Button Hold Pulse

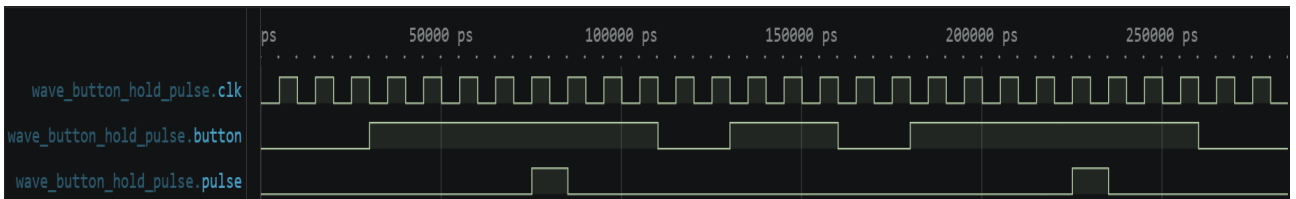


Figure 13: Measured waveform for `button_hold_pulse.sv`

The waveform displays inputs `clk` and `button` and output `pulse`. We observe that:

- **pulse** goes high for a single clock cycle after **button** is pressed for appropriate **HOLD_CYCLES** within **button_hold_detect** module.

3.12 Button Auto Repeat

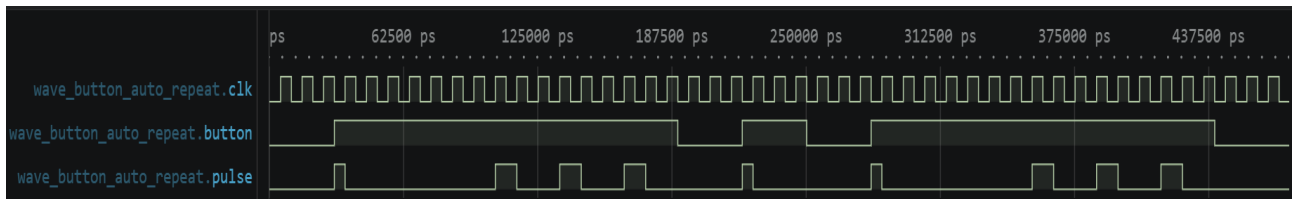


Figure 14: Measured waveform for button_auto_repeat.sv

The waveform displays inputs **clk** and **button** and output **pulse**. We observe that:

- **pulse** goes high for a single clock cycle after **button** is pressed.
- **pulse** train after **button** has been held for **HOLD_CYCLES - REPEAT_CYCLES + 1**.

3.13 Arming Latch

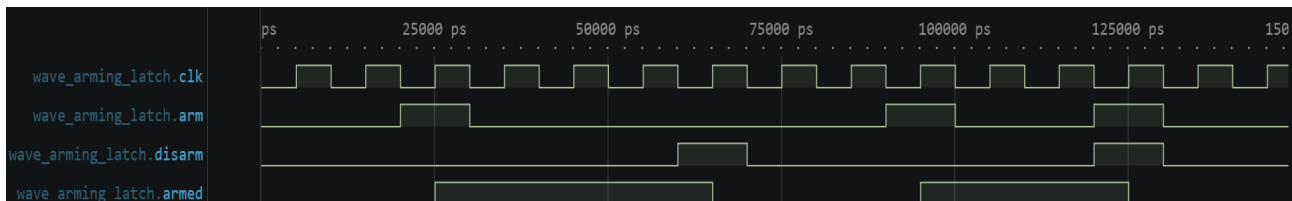


Figure 15: Measured waveform for arming_latch.sv

The waveform displays inputs **clk**, **arm**, and **disarm** and output **armed**. We observe that:

- **armed** goes low next rising edge if **disarm** is high.
- **armed** goes high if **arm** is high and **disarm** is low.

4 Report 3

Report 3

At the start of your report, describe your backup strategy.

Figure 16: Screenshot taken from Digital_Watch_Part_1.pdf

Repository fpga-digital-watch forked and modules completed locally. Changes are then committed and pushed to GitHub after a working session, or after a module has been implemented and tested.

5 Report 4

Report 4

Explain what binary-coded decimal (BCD) is. You may need to research it.

Figure 17: Screenshot taken from Digital_Watch_Part_1.pdf

Binary-coded decimal (BCD) encodes a decimal number into a single binary digit for each place value.

6 Report 5

Report 5

Explain how `localparam CounterWidth = $clog2(CYCLE_COUNT);` works.

Figure 18: Screenshot taken from Digital_Watch_Part_1.pdf

This allows us to set the minimum address width for CounterWidth according to the size of CYCLE_COUNT without the need to state the bit width explicitly. `$2clog(CYCLE_COUNT)` will calculate the smallest power of 2 that will cover memory required for CYCLE_COUNT.

7 Report 6

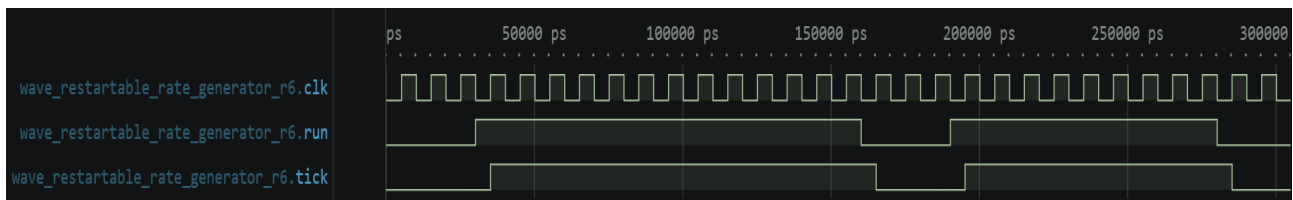


Figure 19: Measured waveform for restartable_rate-generator.sv for case `CYCLE_COUNT = 1`

The waveform displays inputs `clk`, and `run`, and output `tick`. We observe that:

- When `run` is low `tick` is low.
- `tick` follows `run` when high one cycle later. This one cycle delay is expected when implementing a Moore FSM.

8 Report 7

Report 7

Explain why this module is called a PWM generator.

Figure 20: Screenshot taken from Digital_Watch_Part_1.pdf

The module is called PWM generator as it generates a pulse wave and allows as to vary the pulse width with `DUTY_CYCLES`.

9 Report 8

Report 8

In `mod_n_counter`, `rst` takes priority over `enable`. Comment on the benefit of this design choice.

Figure 21: Screenshot taken from Digital_Watch_Part_1.pdf

In our `mod_n_counter` module `rst` takes priority in the sequential logic within the flip-flop block so that the `count` is reset at the next rising clock edge. This ensures count is reset whilst keeping our circuit synchronous.

10 Report 9

Report 9

Explain why the next-state logic may depend on the output `held` without violating the Moore FSM constraint.

Figure 22: Screenshot taken from Digital_Watch_Part_1.pdf

Our state has two variables `held` and `count` and there is no direct path to our output from our input. `held` is a registered output and therefore a part of our current state.

11 Report 10

Report 10

Explain what this module does. Justify your explanation by referring to the generated waveforms, being precise about timing.

Figure 23: Screenshot taken from Digital_Watch_Part_1.pdf

Our `button_hold_pulse` module outputs a one clock cycle pulse once the button runs for the specified `HOLD_CYCLES` within `button_hold_detect` module which has been instantiated in this module.

12 Report 11

Report 11

Explain how `button_hold_pulse` is a Moore FSM despite `rising_edge_detector` being Mealy.

Figure 24: Screenshot taken from Digital_Watch_Part_1.pdf

The module `button_hold_pulse` is a Moore FSM as the input `held` is sent from `button_hold_detect` which is a Moore FSM. Meaning, the input to `button_hold_pulse` can only occur on a rising clock edge.